

Random, String and Math classes



Learning so far ...

- Difference between unstructured, procedural and OOP Languages
- Understood a computing machine and how a program get executed in a machine
- Difference between basic programming languages: C++ and Java (proper OOP Lang.)
- Characteristics of Java – Java environment – program structure
- Packages, static keyword, nested classes, Applet (web apps)
- Characteristics of OOP paradigm
- Classes and Objects
- Access specifiers: Public, private
- Constructors – default and parametrized constructors
- Implicit and Explicit parameters

Accessor and Mutator Methods

- Accessor method: does not change the state of its implicit parameter
`double width = box.getWidth();`
- Mutator method: changes the state of its implicit parameter
`box.translate(15, 25);`

```
Rectangle box = new Rectangle(5, 10, 20, 30);
```

Self Check

- Is the toUpperCase method of the String class an accessor or a mutator?
- Which call to translate is needed to move the box rectangle so that its top-left corner is the origin (0,0)?

Answers

- An accessor—it doesn't modify the original string but returns a new string with uppercase letters
- `box.translate(-5, -10)`

Importing Packages

- Don't forget to include appropriate packages: Java classes are grouped into packages
- Import library classes by specifying the package and class name:
`import java.awt.Rectangle;`
- You don't need to import classes in the `java.lang` package such as String and System

Example

```
import java.awt.Rectangle;
public class MoveTester
{
    public static void main(String[] args)
    {
        Rectangle box = new Rectangle(5, 10, 20, 30);
                                   // Move the rectangle
        box.translate(15, 25);
                                   // Print information about the moved rectangle
        System.out.println("After moving, the top-left corner is:");
        System.out.println(box.getX());
        System.out.println(box.getY());
    }
}
```

Output

- After moving, the top-left corner is:
20
35

Self Check

- The `Random` class is defined in the `java.util` package. What do you need to do in order to use that class in your program?

Answers

- Add the statement `import java.util.Random;` at the top of your program

Random class

- `Random r = new Random();`
- `int i = r.nextInt(int n)` Returns random int ≥ 0 and $< n$
- `int i = r.nextInt()` Returns random int (full range)
- `long i = r.nextLong()` Returns random long (full range)
- `float f = r.nextFloat()` Returns random float ≥ 0.0 and < 1.0
- `double d = r.nextDouble()` Returns random double ≥ 0.0 and < 1.0
- `boolean b = r.nextBoolean()` Returns random bit value (true ,false)
- `double x;`
`x = Math.random();` // assigns random number to x

Object References

- Describe the location of objects
- The **new** operator returns a reference to a new object
Rectangle box = new Rectangle();
- Multiple object variables can **refer** to the same object
Rectangle box = new Rectangle(5, 10, 20, 30);
Rectangle box2 = box;
box2.translate(15, 25);
- Primitive type variables $\neq$ object variables

Self Check

- What is the effect of the assignment `greeting2 = greeting`?
- After calling `greeting2.toUpperCase()`, what are the contents of `greeting` and `greeting2`?

Answers

- Now `greeting` and `greeting2` both refer to the same `String` object.
- Both variables still refer to the same string, and the string has not been modified. Recall that the `toUpperCase` method **constructs a new string** that contains uppercase characters, leaving the original string unchanged.

Method Definition

- *accessSpecifier* *returnType* *methodName*(
 parameterType *parameterName*, . . .)
 {
 method body
 }

Example:

- **public** void deposit(**double** amount)
 { . . .
 }

Constructor Definition

- *accessSpecifier* *ClassName*(*parameterType* *parameterName*, . . .)
 {
 constructor body
 }

Example:

- public BankAccount(double initialBalance)
 {
 . . .
 }

The return Statement

- return *expression*;
or
return;

Example:

- return balance;

Primitive Types

Type	Description	Size
<code>int</code>	The integer type, with range -2,147,483,648 . . . 2,147,483,647	4 bytes
<code>byte</code>	The type describing a single byte, with range -128 . . . 127	1 byte
<code>short</code>	The short integer type, with range -32768 . . . 32767	2 bytes
<code>long</code>	The long integer type, with range -9,223,372,036,854,775,808 . . . 9,223,372,036,854,775,807	8 bytes
<code>double</code>	The double-precision floating-point type, with a range of about $\pm 10^{308}$ and about 15 significant decimal digits	8 bytes
<code>float</code>	The single-precision floating-point type, with a range of about $\pm 10^{38}$ and about 7 significant decimal digits	4 bytes
<code>char</code>	The character type, representing code units in the Unicode encoding scheme	2 bytes
<code>boolean</code>	The type with the two truth values <code>false</code> and <code>true</code>	1 bit

Cast

- *(typeName) expression*

Example:

- `(int) (balance * 100)`

Purpose:

- To convert an expression to a different type

Type Casting

Which of the following are legal?

- `int x = 3.5;`

Illegal: 3.5 is not an int

- `float x = 3;`

Legal: 3 is an int, which is also a Float

- `long i = 3;`

Legal: 3 is an int, which is also a long

- `byte x = 155;`

Illegal: 155 is too big to be a byte (> 127)

- `double d = 3.14159f;`

Legal: 3.14159f is a float, which is also a double

What is “Type Casting”?

- Type casting: automatic conversion of values from one type to another
- e.g. int → double
 - Float → double
 - Int → long
- Type casts can be:
 - Implicit: performed automatically
 - Explicit: programmed by developer

Float range from 1.40129846432481707e-45
to 3.40282346638528860e+38

Implicit Type Casting in Java

- Hierarchy of primitive numerical types

double
float
long
int
short
byte



- Idea

- Higher types have more precision
- Lower types are “subsets” of higher types

- Java only performs implicit **upcasts** (casting from lower to higher type)

e.g. int → double
float → double
int ↗ byte

Explicit Type Casting in Java

- To explicitly cast a value to a type `t`, use `(<t>)` value
 - `int x = (int)3.7;`
 - Assigns value 3 to `x`
 - Reason: `(int)` operator converts double to int by truncation (chopping off decimal) when double is small enough
 - This is an example of **downcasting**
 - `byte bt = (byte) 200;`
 - Assigns value -56 to `bt`
 - Reason: `(byte)` operator “wraps around” values that are too big
- This sounds confusing
 - It is!
 - Rule of thumb:
 - Only use explicit casts when you know what the answer is likely to be
 - Otherwise (e.g. in “overflow” situations) write your own type-conversion routines (we’ll see how later this semester)

Mixed Expressions

- What is result of
`float x = 3 / 4;`
 - x assigned value 0.0F
 - Why?
 - 3, 4 are ints
 - So integer / operation is used, yielding 0, before upcasting is performed
- To get floating point result, use explicit casting
`float x = (float) 3 / (float) 4;`
 - Assigns x the value 0.75F
- Can also do following
`float x = (float) 3 / 4;`
 - Why?
 - `(float) 3` returns a value type float (3.0F)
 - 4 is an int
 - In this case, Java compiler uses upcasting on “lower” type (here, `int`) to obtain values in same type before computing operation

Self Check

- Which are the most commonly used number types in Java?
- When does the cast `(long) x` yield a different result from the call `Math.round(x)`?

Answers

- int and double
- When the fractional part of x is ≥ 0.5

Constants: final

- A **final** variable is a constant
- Once its value has been set, it cannot be changed
- Named constants make programs easier to read and maintain
- Convention: use all-uppercase names for constants

```
final double QUARTER_VALUE = 0.25;  
final double DIME_VALUE = 0.1;
```

Constants: static final

- static is used with class members
- If constant values are needed in several methods, declare them together and tag them as **static** and **final**
- Give static final constants **public** access to enable other classes to use them

```
public class Math  
{
```

```
...
```

```
public static final double E = 2.7182818284590452354;  
public static final double PI = 3.14159265358979323846;  
}
```

```
double circumference = Math.PI * diameter;   (call without object)
```


Syntax : Constant Definition

- In a method:
final typeName variableName = expression ;
- In a class:
accessSpecifier static final typeName variableName = expression;

Example:

- *final double NICKEL_VALUE = 0.05;*
- *public static final double LITERS_PER_GALLON = 3.785;*

Increment, and Decrement

- `items++` is the same as `items = items + 1`
- `items--` subtracts 1 from items
 - `++a;` Let, `a=10`
 - `--a;`
 - `a++;`
 - `a--;`
- 11,10,10,11..... (Top to Bottom)
- 10, 9, 9,10.....(Bottom to Top)

Arithmetic Operations

- $/$ is the division operator
- If both arguments are integers, the result is an integer. The remainder is discarded
 - $7.0 / 4$ yields ? 1.75
 - $7 / 4$ yields ? 1
- Get the remainder with $\%$ (pronounced "modulo")
 $7 \% 4$ is ? 3

The Math class

- Math class: contains methods like `sqrt` and `pow`
- To compute x^n , you write `Math.pow(x, n)`
- However, to compute x^2 it is significantly more efficient simply to compute `x * x`
- To take the square root of a number `x`, use the `Math.sqrt`;
for example,
`Math.sqrt(x)`

Mathematical Methods in Java

<code>Math.sqrt(x)</code>	square root
<code>Math.pow(x, y)</code>	power x^y
<code>Math.exp(x)</code>	e^x
<code>Math.log(x)</code>	natural log
<code>Math.sin(x), Math.cos(x), Math.tan(x)</code>	sine, cosine, tangent (x in radian)
<code>Math.round(x)</code>	closest integer to x
<code>Math.min(x, y), Math.max(x, y)</code>	minimum, maximum

java.lang.Math methods

- E, PI (constants)
- sin(_), cos(_), abs(_), tan(_), ceil(_), floor(_), log(_), max(_,_), min(_,_), pow(_,_), sqrt(_), round(_), random(), toDegrees(_), toRadians(_)

Self Check

- What is the difference between the following two statements?
`final double CM_PER_INCH = 2.54;`
and
`public static final double CM_PER_INCH = 2.54;`
- What is wrong with the following statement?
`double circumference = 3.14 * diameter;`

Answers

- The first definition is used **inside a method**, the second **inside a class**
- (1) You should use a named constant, not 3.14
(2) 3.14 is not an accurate representation of π

Self Check

- What is the value of $1729 / 100$? and $1729 \% 100$?
- What does the following statement compute ?
`double average = s1 + s2 + s3 / 3;` `[s1=10,s2=20,s3=30]`
- What is the value of `Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2))` in mathematical notation?

Answers

- 17 and 29
- 40: Only s3 is divided by 3. To get the correct result, use parentheses. Moreover, if s1, s2, and s3 are integers, you must divide by 3.0 to avoid integer division:
 $(s1 + s2 + s3) / 3.0$

Calling Static Methods

- A static method does not operate on an object
Double x = 4;
Double root = x.sqrt(); // Error
- Naming convention: Classes start with an uppercase letter; objects start with a lowercase letter
Math
System.out

Static Method Call

- *ClassName. methodName(parameters)*

Example:

- `Math.sqrt(4)`

Purpose:

- To invoke a static method (a method that does not operate on an object) and supply its parameters

double x=10.0

Self Check

- Why can't you call `x.pow(y)` to compute x^y ?
- Is the call `System.out.println(4)` a static method call?

Answers

- x is a number, not an object, and you cannot invoke methods on numbers
- No—the `println` method is called on the object `System.out`

Strings

- A string is a sequence of characters
- Strings are objects of the String class
- String constants:
 "Hello, World!"
- String variables:
 String message = "Hello, World!";
- String length:
 int n = message.length();
- Empty string: ""

Concatenation

- Use the + operator:

```
String name = "Dave";  
String message = "Hello, " + name;  
// message is "Hello, Dave"
```

- If one of the arguments of the + operator is a string, the other is converted to a string

```
String a = "Agent";  
int n = 7;  
String bond = a + n; // bond is Agent7
```

Converting between Strings and Numbers

- Convert to number:

```
int n = Integer.parseInt(str);  
double x = Double.parseDouble(str);
```

- Convert to string:

```
String str = "" + n;  
str = Integer.toString(n);
```

Substrings

- String greeting = "Hello, World!";
String sub = **greeting.substring(0, 5);**
// sub is "Hello"
- Supply start and end positions
- First position is at 0

Self Check

- Assuming the String variable `s` holds the value "Super", what is the effect of the assignment `s = s + s.length()` ?
- Assuming the String variable `river` holds the value "Mississippi", what is the value of
`river.substring(1, 2)?` and
`river.substring(2, river.length() - 3)?`

Answers

- `s` is set to the string Super5
- The strings "i" and
- "ssissi"

New learning

- Accessor and Mutator Methods
- Random class – Util package
- Type casting
- Constants – Final and static final
- Static methods
- Math class methods
- Converting strings and numbers, etc.